

神经网络控制作业报告

船舶航向跟踪

姓名：罗习伦 学号：3230105342

2026 年 6 月 1 日

摘要

目录

1 建立 USV 的神经网络辨识模型，并分析辨识结果的精度	3
1.1 系统概览	3
1.2 神经网络辨识模型的建立	3
1.3 测试与定量评价	3
2 直接神经网络控制	5
2.1 系统概览	5
2.2 基于梯度下降的在线权值更新律推导	5
2.2.1 输出层参数更新	5
2.2.2 隐藏层参数更新	6
2.2.3 系统 Jacobian 估计	6
2.2.4 总结	7
2.3 测试结果与分析	7
2.3.1 评价指标	7
2.3.2 符号近似法	8
2.3.3 神经网络估计法	9
2.3.4 分析	9
3 神经网络 PID 控制系统	9
3.1 系统概览	9
3.2 基于梯度下降的在线权值更新律推导	10
3.2.1 PID 控制律与网络结构	10
3.2.2 输出层参数更新	11
3.2.3 隐藏层参数更新	11
3.2.4 系统 Jacobian 估计	11

3.3	测试结果与分析	12
3.3.1	符号近似法	12
3.3.2	神经网络估计法	12
3.3.3	分析	13
4	针对仿射非线性 USV 模型的 RBF 神经网络自适应控制器	13
4.1	系统概览	13
4.2	RBF 神经网络结构, 控制律与在线权值更新律推导	14
4.2.1	全参数在线学习的 RBF 逼近器	14
4.2.2	跟踪误差定义与控制律	14
4.2.3	学习信号 (增强误差) 与性能指标	14
4.2.4	全参数在线更新律 (反向传播形式)	15
4.2.5	方案特点 (与仅调权值的 RBF/模糊自适应的区别)	15
4.2.6	控制器设计	16
4.3	测试结果与分析	16
4.3.1	分析	16
5	总结	17
6	附录	17
6.1	文件清单	17

1 建立 USV 的神经网络辨识模型，并分析辨识结果的精度

1.1 系统概览

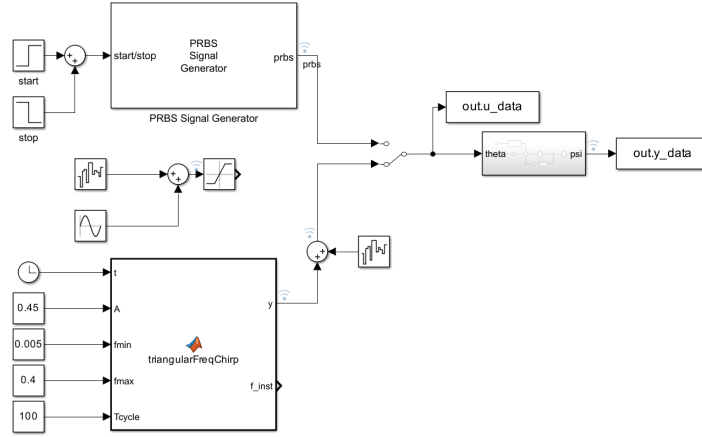


图 1: 系统概览图

本系统的核心是 USV 系统的激励信号生成。在本任务及后续任务的实验过程中，我先后尝试了二进制伪随机数、带白噪声的固定频率正弦信号和带白噪声的扫频信号三种激励信号，发现虽然神经网络在不同激励下均能满足拟合的激励要求，但是只有扫频信号能够充分激发系统的动态特性，从而使得神经网络能够适应更广泛的输入情况。因此在后续的实验中均采用带白噪声的扫频信号作为激励信号。

1.2 神经网络辨识模型的建立

在建立神经网络辨识模型时，我采用了 MATLAB 自带的 `feedforwardnet` 函数来创建前馈神经网络。在实验初期，我使用的系统信号输入阶数为 2，即输入信号为 $u(k-1), y(k-1), y(k-2)$ ，输出信号为 $y(k)$ 。但是在实际训练过程中发现该模型的辨识精度并不理想，而直接增加隐藏层神经元数会导致过拟合，预测效果更差。考虑到 USV 的实际系统模型是一个二阶系统，因此将系统信号输入阶数提升至 3，即输入信号为 $u(k-1), u(k-2), y(k-1), y(k-2), y(k-3)$ ，输出信号仍为 $y(k)$ ，从而提升了神经网络辨识模型的预测效果。

1.3 测试与定量评价

为定量评估所辨识的网络函数 `USV_net_function`（延迟阶数 $m = 5$ ，即总输入维度为 6），将其与 Simulink 真实被控对象在同采样周期下进行一阶预测（one-step-ahead）对比，统计误差指标如表1。

表 1: 辨识模型在不同激励下的一阶预测定量结果（排除初始延迟段）

激励	MSE	RMSE	MAE	MaxAbsError	R^2	Fit(%)
PRBS (0.05–0.4)	0.0962	0.3102	0.1556	1.8030	0.9272	73.0140
扫频 (0.05–0.4)	0.0015	0.0391	0.0166	0.3602	0.9988	96.6006

结果表明：扫频激励下模型拟合精度显著高于 PRBS (R^2 与 Fit 更高、误差显著更小)，说明扫频信号能更充分激发系统动态特性，从而提升辨识泛化能力。

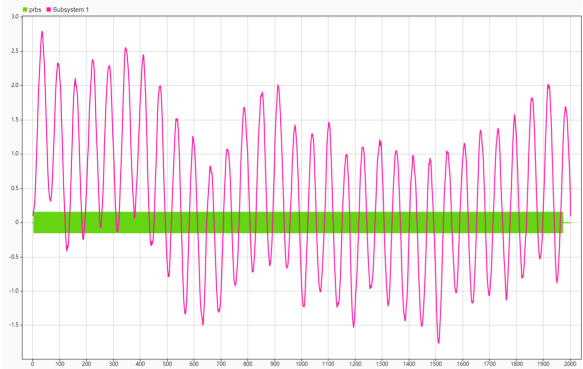


图 2: PRBS 激励及输出

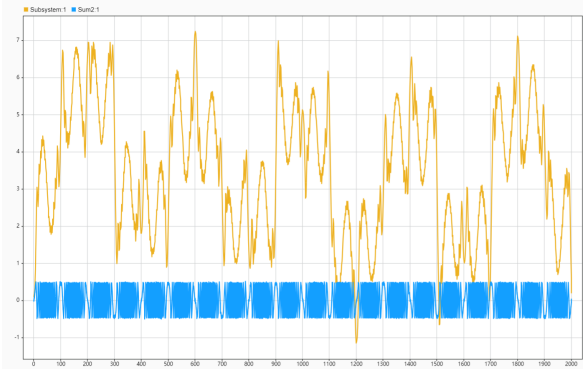


图 3: 扫频激励及输出

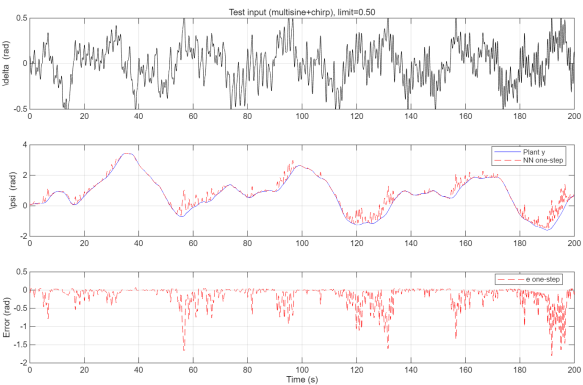


图 4: PRBS 辨识模型测试结果

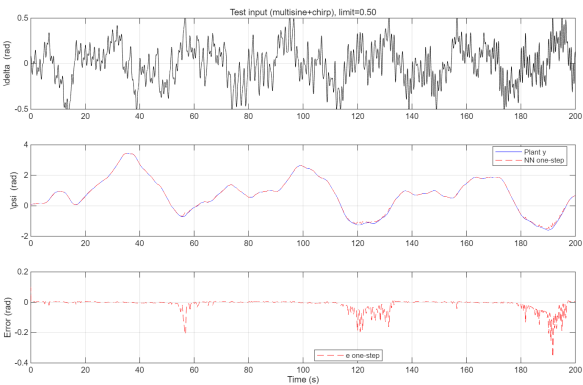


图 5: 扫频信号辨识模型测试结果

2 直接神经网络控制

2.1 系统概览

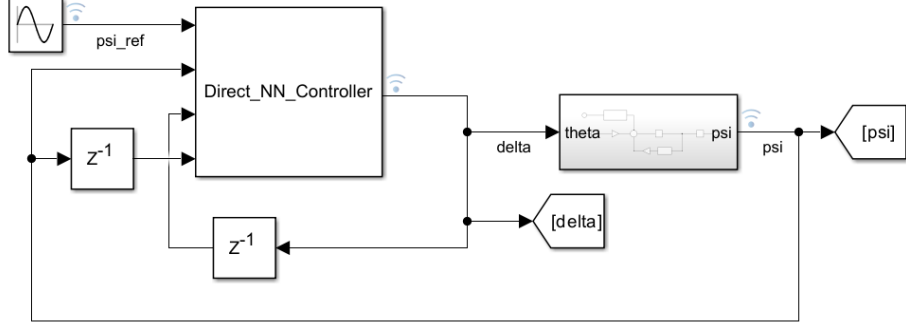


图 6: 直接神经网络控制 Simulink 系统概览图

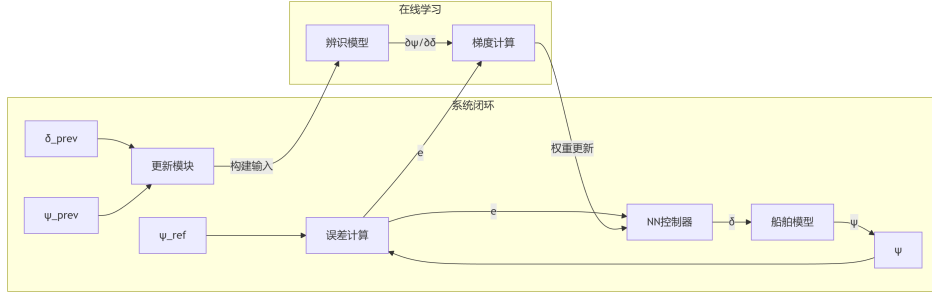


图 7: 直接神经网络控制系统功能框图

我设计了一个基于 BP 神经网络的直接控制系统，其中神经网络控制器直接根据参考航向与实际航向的误差及其历史值计算控制舵角。采用梯度下降法在线更新控制器权值，更新律依赖于系统 Jacobian（即系统输出对控制输入的灵敏度）的估计。在仿真中，我对比了两种 Jacobian 估计方法：符号近似法（仅使用符号函数简化计算）和神经网络估计法（通过神经网络辨识模型来近似系统动态，从而数值计算 Jacobian）。

2.2 基于梯度下降的在线权值更新律推导

为实现直接神经网络控制器的在线自适应，定义瞬时性能指标（损失函数）

$$J(k) = \frac{1}{2} e^2(k) = \frac{1}{2} [\psi_{\text{ref}}(k) - \psi(k)]^2, \quad (1)$$

其中 $e(k)$ 为航向跟踪误差， $\psi_{\text{ref}}(k)$ 为参考航向， $\psi(k)$ 为实际航向，控制器输出舵角为 $\delta(k)$ 。

2.2.1 输出层参数更新

输出层权重 W_o 的梯度为

$$\frac{\partial J(k)}{\partial W_o} = \frac{\partial J}{\partial e} \frac{\partial e}{\partial \psi} \frac{\partial \psi}{\partial \delta} \frac{\partial \delta}{\partial W_o} = e(k) \cdot (-1) \cdot J_s(k) \cdot h(k) = -e(k) J_s(k) h(k), \quad (2)$$

其中 $J_s(k) = \frac{\partial \psi(k)}{\partial \delta(k)}$ 为系统 Jacobian, $h(k)$ 为隐藏层输出。输出层偏置 b_o 的梯度为

$$\frac{\partial J(k)}{\partial b_o} = -e(k)J_s(k). \quad (3)$$

采用带动量项的梯度下降更新:

$$W_o(k+1) = W_o(k) + \eta e(k)J_s(k)h(k) + \alpha \Delta W_o(k-1), \quad (4)$$

$$b_o(k+1) = b_o(k) + \eta e(k)J_s(k) + \alpha \Delta b_o(k-1), \quad (5)$$

其中 η 为学习率, α 为动量系数。

2.2.2 隐藏层参数更新

令控制器输入向量

$$X(k) = [e(k), e(k-1), e(k-2), 1]^T. \quad (6)$$

隐藏层权重 W_i 的梯度由链式法则得到

$$\frac{\partial J(k)}{\partial W_i} = -e(k)J_s(k)W_o(k)[1 - h^2(k)]X(k), \quad (7)$$

其中 $[1 - h^2(k)]$ 为 $\tanh(\cdot)$ (tansig) 导数项; 隐藏层偏置梯度为

$$\frac{\partial J(k)}{\partial b_i} = -e(k)J_s(k)W_o(k)[1 - h^2(k)]. \quad (8)$$

相应在线更新律为

$$W_i(k+1) = W_i(k) + \eta e(k)J_s(k)W_o(k)[1 - h^2(k)]X(k) + \alpha \Delta W_i(k-1), \quad (9)$$

$$b_i(k+1) = b_i(k) + \eta e(k)J_s(k)W_o(k)[1 - h^2(k)] + \alpha \Delta b_i(k-1). \quad (10)$$

2.2.3 系统 Jacobian 估计

更新律关键项为 $J_s(k) = \frac{\partial \psi(k)}{\partial \delta(k)}$ 。仿真中对比两种估计方式:

(1) 符号近似法:

$$J_s(k) \approx \text{sign}\left(\frac{\psi(k)_{\delta+\Delta} - \psi(k)_\delta}{\Delta}\right), \quad \Delta = 0.05. \quad (11)$$

(2) 辨识网络数值差分法:

$$J_s(k) \approx 50 \cdot \frac{\psi(k)_{\delta+\Delta} - \psi(k)_\delta}{\Delta}, \quad J_s(k) \in [-10, 10], \quad (12)$$

其中 $\psi(k)_\delta$ 与 $\psi(k)_{\delta+\Delta}$ 由辨识网络 f_{ident} 计算, 例如

$$\psi(k)_\delta = f_{\text{ident}}([\delta(k-1), \psi(k-1), \psi(k-2), \dots, \psi(k-m)]^T). \quad (13)$$

2.2.4 总结

定义误差项

$$\delta_o(k) = e(k)J_s(k), \quad \delta_i(k) = [1 - h^2(k)] \cdot W_o^T(k) \cdot \delta_o(k), \quad (14)$$

则更新可写为

$$W_o(k+1) = W_o(k) + \eta \delta_o(k) h^T(k) + \alpha \Delta W_o(k-1), \quad b_o(k+1) = b_o(k) + \eta \delta_o(k) + \alpha \Delta b_o(k-1), \quad (15)$$

$$W_i(k+1) = W_i(k) + \eta \delta_i(k) X^T(k) + \alpha \Delta W_i(k-1), \quad b_i(k+1) = b_i(k) + \eta \delta_i(k) + \alpha \Delta b_i(k-1). \quad (16)$$

2.3 测试结果与分析

反复尝试发现，控制效果与系统辨识神经网络的结构密切相关。系统信号输入阶数为 2 时，估计精度不足，残差较大；提升至阶数 3 时，估计精度显著提升，控制效果最佳，且控制器输出不饱和；再提升至阶数 5 时，系统收敛很慢，且控制器输出饱和，效果变差。最终选用阶数 3 的辨识网络进行直接神经网络控制实验。

2.3.1 评价指标

为保证实验结果可量化,本文所有指标均按照如下方式计算(程序实现见 `analyze_global.m`):

- **最大跟踪误差 (Maximum Tracking Error):**

$$e_{\max} = \max_{t \in [0, T_{\text{sim}}]} |\psi(t) - \psi_{\text{ref}}(t)|.$$

- **均方根误差 (Root Mean Square Error):**

$$e_{\text{RMS}} = \sqrt{\frac{1}{T_{\text{sim}}} \int_0^{T_{\text{sim}}} [\psi(t) - \psi_{\text{ref}}(t)]^2 dt},$$

对应脚本中的 `RMS_error`。

- **平均绝对误差 (Mean Absolute Error):**

$$e_{\text{MAE}} = \frac{1}{T_{\text{sim}}} \int_0^{T_{\text{sim}}} |\psi(t) - \psi_{\text{ref}}(t)| dt,$$

对应脚本中的 `MAE_error`。

- **跟踪误差带内时间比例 (In-Band Time Ratio):** 定义瞬时跟踪误差 $e(t) = \psi(t) - \psi_{\text{ref}}(t)$ 和误差带

$$\text{band} = \max(0.05, 0.05 \times \max_{t \in [0, T_{\text{sim}}]} |\psi_{\text{ref}}(t)|).$$

则带内时间比例为

$$R_{\text{band}} = \frac{1}{T_{\text{sim}}} \int_0^{T_{\text{sim}}} \mathbf{1}_{\{|e(t)| \leq \text{band}\}} dt,$$

其中 $\mathbf{1}_{\{\cdot\}}$ 为指示函数。

- **归一化误差指标 (Normalized Error Metric):** 为消除参考信号幅值影响, 定义归一化均方根误差

$$e_{\text{nRMS}} = \frac{e_{\text{RMS}}}{\sqrt{\frac{1}{T_{\text{sim}}} \int_0^{T_{\text{sim}}} \psi_{\text{ref}}^2(t) dt}}.$$

- **控制输入平滑性 (Control Smoothness):** 由控制输入 $\delta(t)$ 离散序列 $\delta[k]$ 给出

$$S = \frac{1}{N-1} \sum_{k=1}^{N-1} |\delta[k+1] - \delta[k]|,$$

并计算最大输入变化率

$$S_{\text{max}} = \max_{t \in [0, T_{\text{sim}}]} \left| \frac{d\delta}{dt} \right|,$$

对应脚本 `delta_smooth` 与 `Control Derivative`。

- **控制能量 (Control Energy):** 分别定义 L_1 能量与 L_2 能量:

$$E_1 = \int_0^{T_{\text{sim}}} |\delta(t)| dt, \quad E_2 = \sqrt{\int_0^{T_{\text{sim}}} \delta^2(t) dt},$$

对应 `Control_L1` 与 `Control_L2`。

- **控制变化率能量 (Control Variation Energy):**

$$V = \int_0^{T_{\text{sim}}} \left(\frac{d\delta}{dt} \right)^2 dt.$$

备注: 对于动态参考信号, 传统阶跃响应指标 (如超调、调节时间) 不再适用, 故采用上述通用跟踪性能指标。其中带内时间比例 R_{band} 可反映系统在误差带内的驻留时间比例, 归一化误差 e_{nRMS} 可用于比较不同幅值参考信号下的控制性能。

2.3.2 符号近似法

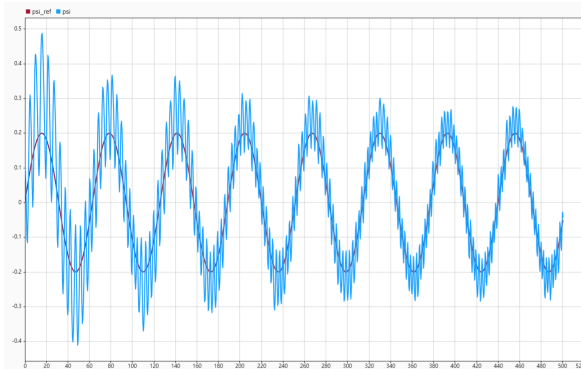


图 8: 符号近似法控制结果

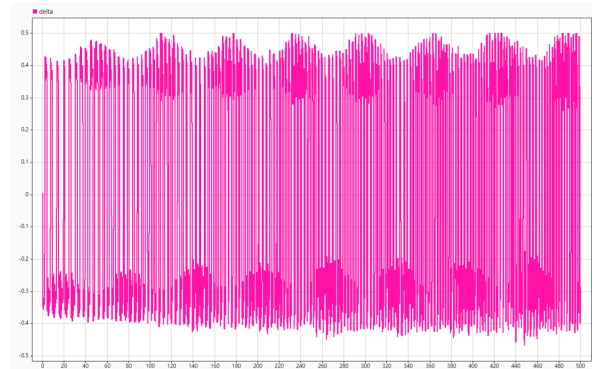


图 9: 符号近似法控制输入

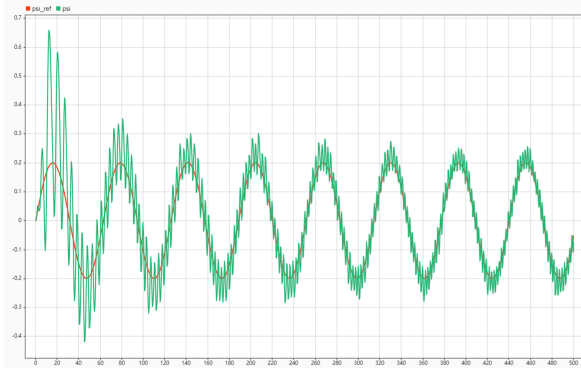


图 10: 神经网络估计法控制结果

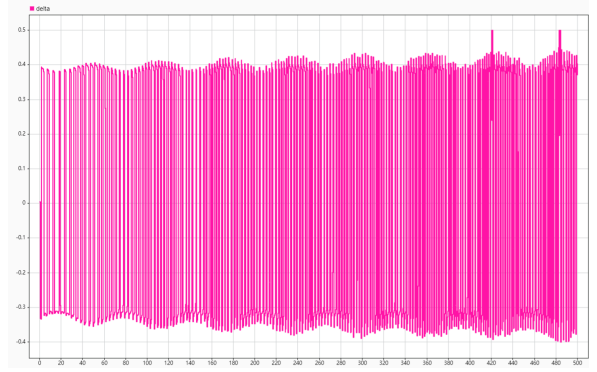


图 11: 神经网络估计法控制输入

2.3.3 神经网络估计法

实验过程中发现，使用 $\frac{\partial \psi}{\partial u} \approx \frac{\psi(\mathbf{u} + \delta \mathbf{u}) - \psi(\mathbf{u})}{\delta \mathbf{u}}$ 估计得到的 Jacobian 数值非常小，导致权值更新过慢，系统响应迟钝。通过放大 Jacobian 数值（乘以 jacobian_gain）后，控制效果显著提升，系统能够较快收敛，且控制器输出不饱和。

表 2: 直接神经网络控制性能指标对比汇总

指标	符号近似法	神经网络估计法
最大跟踪误差 $e_{\max}$ (rad)	0.2882	0.2438
均方根误差 e_{RMS} (rad)	0.0767	0.0602
平均绝对误差 e_{MAE} (rad)	0.0611	0.0470
归一化均方根误差 e_{nRMS}	0.5412	0.4246
带内时间比例 R_{band}	47.79%	64.46%
控制平滑性 S (rad/sample)	0.0276	0.0059
控制 L_1 能量 E_1	159.3832	164.6546
控制 L_2 能量 E_2	7.4265	7.5458
控制变化率能量 V	4944.6985	1200.8135

2.3.4 分析

从实验结果可以看出，基于神经网络估计法的 Jacobian 计算方式在多数跟踪性能指标上均优于符号近似法，尤其是在均方根误差、平均绝对误差和带内时间比例等关键指标上表现更佳，但是收敛速率较慢。此外，神经网络估计法的控制输入更加平滑，变化率能量显著降低，说明该方法在保证控制性能的同时，也提升了控制输入的平滑性，有助于实际系统的稳定运行。

3 神经网络 PID 控制系统

3.1 系统概览

我设计了一个基于 BP 神经网络的 PID 控制系统，其中神经网络控制器根据参考航向与实际航向的误差及其历史值计算 PID 控制增量。采用梯度下降法在线更新控制器权值，更新律依

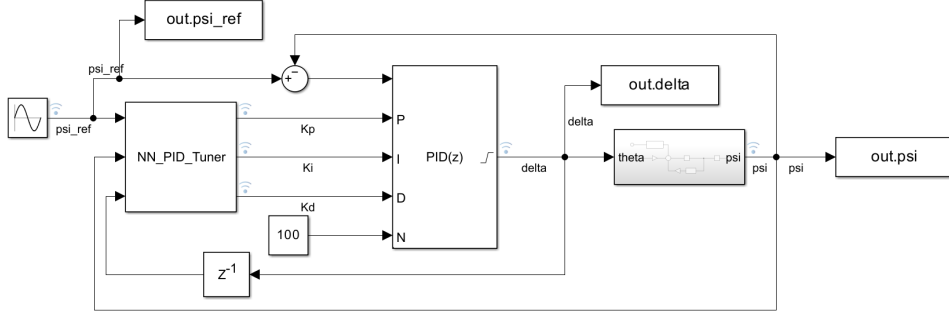


图 12: 神经网络 PID 控制 Simulink 系统概览图

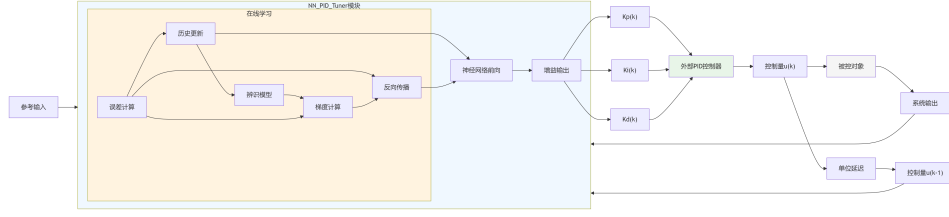


图 13: 神经网络 PID 控制系统功能框图

赖于系统 Jacobian 的估计。在仿真中，同样对比了符号近似法和神经网络估计法两种 Jacobian 估计方式。

3.2 基于梯度下降的在线权值更新律推导

神经网络 PID 控制器采用增量式 PID 控制律，同时通过 BP 神经网络在线自适应调节 PID 三个增益参数 K_p, K_i, K_d 。定义瞬时性能指标

$$J(k) = \frac{1}{2}e^2(k) = \frac{1}{2}[\psi_{\text{ref}}(k) - \psi(k)]^2. \quad (17)$$

3.2.1 PID 控制律与网络结构

采用增量式 PID 控制律

$$u(k) = u(k-1) + K_p(k)[e(k) - e(k-1)] + K_i(k)e(k) + K_d(k)[e(k) - 2e(k-1) + e(k-2)], \quad (18)$$

其中增益参数 $K_p(k), K_i(k), K_d(k)$ 由三层 BP 神经网络输出：

- 输入层： $X(k) = [e(k), e(k-1), e(k-2), 1]^T$ ；
- 隐藏层： $h(k) = \tanh(W_i(k)X(k) + b_i(k))$, $\tanh(x) = \frac{2}{1+e^{-2x}} - 1$ ；
- 输出层： $g(k) = \sigma(W_o(k)h(k) + b_o(k))$, $\sigma(x) = \frac{1}{1+e^{-x}}$ ；
- 增益约束：

$$K_p(k) = K_{p,\max} g_1(k), \quad K_i(k) = K_{i,\max} g_2(k), \quad K_d(k) = K_{d,\max} g_3(k), \quad (19)$$

其中 $g = [g_1, g_2, g_3]^T$ ，保证增益参数非负且有界。

3.2.2 输出层参数更新

控制输入对各增益参数的灵敏度为

$$\frac{\partial u(k)}{\partial K_p(k)} = e(k) - e(k-1), \quad (20)$$

$$\frac{\partial u(k)}{\partial K_i(k)} = e(k), \quad (21)$$

$$\frac{\partial u(k)}{\partial K_d(k)} = e(k) - 2e(k-1) + e(k-2). \quad (22)$$

定义系统 Jacobian $J_s(k) = \frac{\partial \psi(k)}{\partial u(k)}$ 。对于第 j 个输出神经元 (对应 K_p, K_i, K_d)，由链式法则得到

$$\frac{\partial J(k)}{\partial \text{net}_{o,j}(k)} = \frac{\partial J}{\partial e} \frac{\partial e}{\partial \psi} \frac{\partial \psi}{\partial u} \frac{\partial u}{\partial K_j} \frac{\partial K_j}{\partial g_j} \frac{\partial g_j}{\partial \text{net}_{o,j}} = e(k) \cdot (-1) \cdot J_s(k) \cdot \frac{\partial u}{\partial K_j} \cdot K_{j,\max} \cdot g_j(1 - g_j), \quad (23)$$

其中 $\frac{\partial g_j}{\partial \text{net}_{o,j}} = g_j(1 - g_j)$ 为 sigmoid 导数。定义输出层局部梯度

$$\delta_{o,p}(k) = e(k) J_s(k) [e(k) - e(k-1)] K_{p,\max} g_1(k) [1 - g_1(k)], \quad (24)$$

$$\delta_{o,i}(k) = e(k) J_s(k) e(k) K_{i,\max} g_2(k) [1 - g_2(k)], \quad (25)$$

$$\delta_{o,d}(k) = e(k) J_s(k) [e(k) - 2e(k-1) + e(k-2)] K_{d,\max} g_3(k) [1 - g_3(k)], \quad (26)$$

即

$$\delta_o(k) = \begin{bmatrix} \delta_{o,p}(k) \\ \delta_{o,i}(k) \\ \delta_{o,d}(k) \end{bmatrix}. \quad (27)$$

输出层权值更新律为

$$W_o(k+1) = W_o(k) + \eta \delta_o(k) h^T(k) + \alpha \Delta W_o(k-1), \quad (28)$$

$$b_o(k+1) = b_o(k) + \eta \delta_o(k) + \alpha \Delta b_o(k-1). \quad (29)$$

3.2.3 隐藏层参数更新

隐藏层局部梯度由反向传播得到

$$\delta_h(k) = [1 - h^2(k)] \odot [W_o^T(k) \delta_o(k)], \quad (30)$$

其中 $\odot$ 为逐元素乘法， $[1 - h^2(k)]$ 为 $\tanh$ 导数项。隐藏层权值更新律为

$$W_i(k+1) = W_i(k) + \eta \delta_h(k) X^T(k) + \alpha \Delta W_i(k-1), \quad (31)$$

$$b_i(k+1) = b_i(k) + \eta \delta_h(k) + \alpha \Delta b_i(k-1). \quad (32)$$

3.2.4 系统 Jacobian 估计

与直接神经网络控制类似，系统 Jacobian $J_s(k) = \frac{\partial \psi(k)}{\partial u(k)}$ 采用两种估计方法：

(1) 符号近似法：

$$J_s(k) \approx \text{sign} \left(\frac{\psi(k)_{u+\Delta} - \psi(k)_u}{\Delta} \right), \quad \Delta = 0.05. \quad (33)$$

(2) 辨识网络数值差分法:

$$J_s(k) \approx 50 \cdot \frac{\psi(k)_{u+\Delta} - \psi(k)_u}{\Delta}, \quad J_s(k) \in [-10, 10], \quad (34)$$

其中 $\psi(k)_u$ 与 $\psi(k)_{u+\Delta}$ 由辨识网络计算。

3.3 测试结果与分析

使用系统信号输入阶数为 3，隐藏层神经元数为 6 的辨识网络进行神经网络 PID 控制实验，经过多次调试，最终选用学习率 $\eta = 0.1$ 。

3.3.1 符号近似法

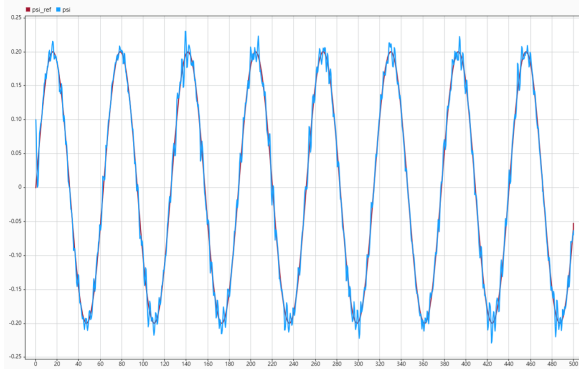


图 14: 符号近似法控制结果

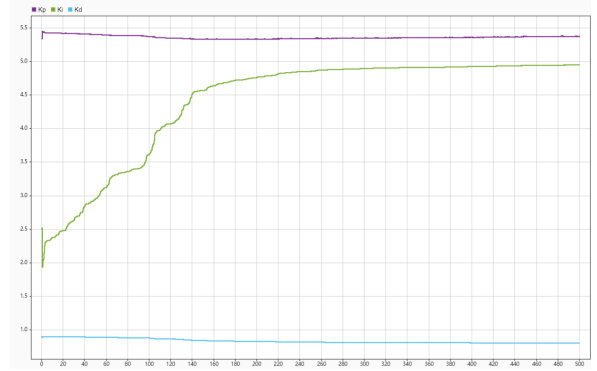


图 15: 符号近似法控制器系数变化

3.3.2 神经网络估计法

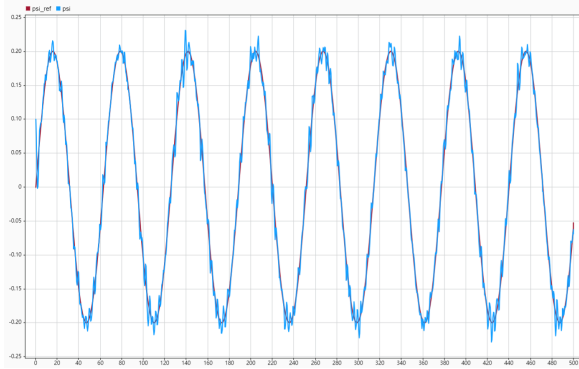


图 16: 神经网络估计法控制结果

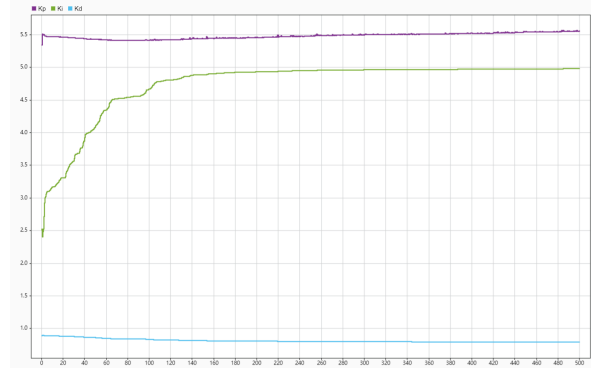


图 17: 神经网络估计法控制器系数变化

表 3: 神经网络 PID 控制性能指标对比汇总

指标	符号近似法	神经网络估计法
最大跟踪误差 $e_{\max}$ (rad)	0.1000	0.1000
均方根误差 e_{RMS} (rad)	0.0113	0.0114
平均绝对误差 e_{MAE} (rad)	0.0086	0.0087
归一化均方根误差 e_{nRMS}	0.0796	0.0803
带内时间比例 R_{band}	99.88%	99.88%
控制平滑性 S (rad/sample)	0.0011	0.0012
控制 L_1 能量 E_1	47.2299	47.5518
控制 L_2 能量 E_2	2.5108	2.5382
控制变化率能量 V	10.3629	10.9244

3.3.3 分析

从实验结果可以看出，神经网络 PID 控制器在两种 Jacobian 估计方式下均表现出优异的跟踪性能，误差指标均非常低，且带内时间比例接近 100%。两种方法的性能指标差异较小，均能实现高精度的航向跟踪控制。 K_I 参数在控制过程中有较大增幅，有效减小了稳态误差。

4 针对仿射非线性 USV 模型的 RBF 神经网络自适应控制器

4.1 系统概览

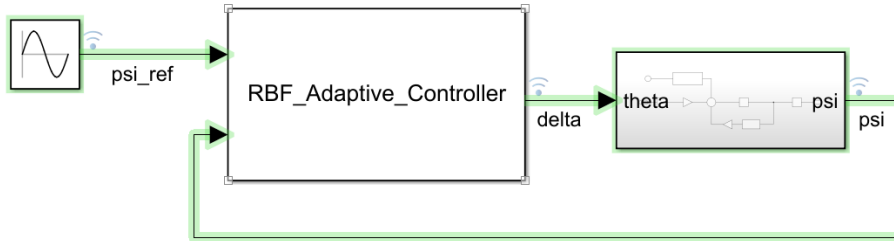


图 18: RBF 神经网络自适应控制 Simulink 系统概览图

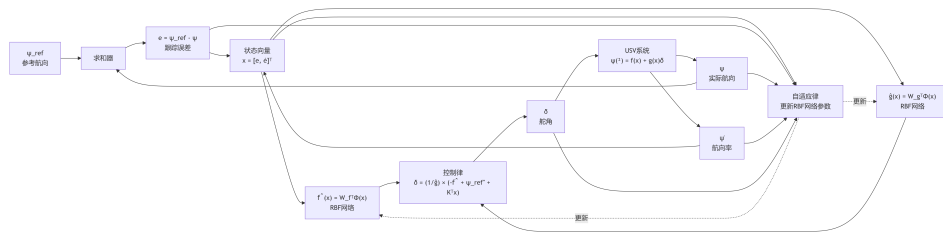


图 19: RBF 神经网络自适应控制系统功能框图

我设计了一个基于径向基函数（RBF）神经网络的自适应控制系统，用于控制仿射非线性

USV 模型的航向。该控制器利用 Lyapunov 方法推导在线权值更新律，避免了对系统 Jacobian 的依赖，从而提升了控制系统的鲁棒性和适应性。

4.2 RBF 神经网络结构，控制律与在线权值更新律推导

RBF 神经网络由输入层、隐藏层和输出层组成。输入层接受误差及其历史值，隐藏层采用高斯径向基函数作为激活函数，输出层为线性组合。

4.2.1 全参数在线学习的 RBF 逼近器

考虑 USV 航向的仿射非线性模型写成

$$\ddot{\psi} = f(\mathbf{x}) + g(\mathbf{x}) \delta, \quad \mathbf{x} = [\psi, \dot{\psi}]^T, \quad (35)$$

其中 $f(\cdot), g(\cdot)$ 未知且 $g(\mathbf{x})$ 有界并满足不变号（便于实现 $\hat{g}^{-1}$ ）。

为更忠实体现 RBF 网络的本质，采用两个 RBF 网络分别逼近 f 与 g ，并令二者共享隐层中心与宽度，仅输出层权值不同：

$$\hat{f}(\mathbf{x} | \Theta_f) = \sum_{j=1}^N w_{f_j} \phi_j(\mathbf{x}; \mathbf{c}_j, \sigma_j) = \mathbf{W}_f^T \Phi(\mathbf{x}; \mathbf{C}, \sigma), \quad (36)$$

$$\hat{g}(\mathbf{x} | \Theta_g) = \sum_{j=1}^N w_{g_j} \phi_j(\mathbf{x}; \mathbf{c}_j, \sigma_j) = \mathbf{W}_g^T \Phi(\mathbf{x}; \mathbf{C}, \sigma), \quad (37)$$

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{2\sigma_j^2}\right), \quad \Phi = [\phi_1, \dots, \phi_N]^T. \quad (38)$$

因此全部可调参数为

$$\Theta = \{\mathbf{W}_f, \mathbf{W}_g, \mathbf{C}, \sigma\}, \quad \mathbf{C} = [\mathbf{c}_1, \dots, \mathbf{c}_N], \quad \sigma = [\sigma_1, \dots, \sigma_N]^T.$$

4.2.2 跟踪误差定义与控制律

定义二阶跟踪误差向量

$$\mathbf{e} = \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} = \begin{bmatrix} \psi - \psi_{\text{ref}} \\ \dot{\psi} - \dot{\psi}_{\text{ref}} \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} k_1 \\ k_2 \end{bmatrix}. \quad (39)$$

采用基于函数逼近的反馈线性化形式控制律

$$\delta = \frac{1}{\hat{g}(\mathbf{x}) + \varepsilon_g} \left[-\hat{f}(\mathbf{x}) + \ddot{\psi}_{\text{ref}} - \mathbf{K}^T \mathbf{e} \right] \quad (40)$$

其中 $\varepsilon_g > 0$ 为小正数以避免除零，并可对 δ 做饱和约束。

4.2.3 学习信号（增强误差）与性能指标

选取稳定矩阵

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -k_1 & -k_2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

并令 $\mathbf{P} > 0$ 为 Lyapunov 方程

$$\mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A} = -\mathbf{Q}, \quad \mathbf{Q} > 0 \quad (41)$$

的解。定义用于驱动学习的增强误差（虚拟误差）

$$s = \mathbf{e}^T \mathbf{P} \mathbf{b} \quad (42)$$

并以瞬时指标

$$J(t) = \frac{1}{2} s^2(t) \quad (43)$$

为统一的在线学习目标。采用梯度下降思想

$$\dot{\Theta} = -\Gamma \frac{\partial J}{\partial \Theta} = -\Gamma s \frac{\partial s}{\partial \Theta}, \quad \Gamma > 0. \quad (44)$$

将误差对参数的敏感性近似为网络输出对参数的敏感性（体现“误差反传”思想）：

$$\frac{\partial s}{\partial \Theta} \propto \frac{\partial}{\partial \Theta} (\hat{f}(\mathbf{x}) + \hat{g}(\mathbf{x}) \delta) = \frac{\partial \hat{f}}{\partial \Theta} + \delta \frac{\partial \hat{g}}{\partial \Theta}. \quad (45)$$

4.2.4 全参数在线更新律（反向传播形式）

(1) 输出层权值更新：由于 $\frac{\partial \hat{f}}{\partial \mathbf{W}_f} = \Phi$ 、 $\frac{\partial \hat{g}}{\partial \mathbf{W}_g} = \Phi$ ，得到

$$\dot{\mathbf{W}}_f = -\gamma_{wf} s \Phi(\mathbf{x}), \quad \dot{\mathbf{W}}_g = -\gamma_{wg} s \Phi(\mathbf{x}) \delta \quad (46)$$

(2) 隐层中心更新：对第 j 个基函数，有

$$\frac{\partial \phi_j}{\partial \mathbf{c}_j} = \phi_j \frac{(\mathbf{x} - \mathbf{c}_j)}{\sigma_j^2}, \quad \frac{\partial \hat{f}}{\partial \mathbf{c}_j} = w_{fj} \frac{\partial \phi_j}{\partial \mathbf{c}_j}, \quad \frac{\partial \hat{g}}{\partial \mathbf{c}_j} = w_{gj} \frac{\partial \phi_j}{\partial \mathbf{c}_j}.$$

由共享中心带来的“责任分配”，中心更新律写为

$$\dot{\mathbf{c}}_j = -\gamma_c s \frac{\phi_j(\mathbf{x})}{\sigma_j^2} (\mathbf{x} - \mathbf{c}_j) (w_{fj} + w_{gj} \delta) \quad (47)$$

(3) 隐层宽度更新：有

$$\frac{\partial \phi_j}{\partial \sigma_j} = \phi_j \frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{\sigma_j^3},$$

因此宽度更新律为

$$\dot{\sigma}_j = -\gamma_\sigma s \phi_j(\mathbf{x}) \frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{\sigma_j^3} (w_{fj} + w_{gj} \delta) \quad (48)$$

工程实现中可加入投影/截断以保证 $\sigma_j \in [\sigma_{\min}, \sigma_{\max}]$ ，并对 $\hat{g}(\mathbf{x})$ 做下界约束以避免控制奇异。

4.2.5 方案特点（与仅调权值的 RBF/模糊自适应的区别）

上述自适应律由同一指标 $J = \frac{1}{2} s^2$ 通过链式法则推得，使误差 s 统一反传到 $\mathbf{W}_f, \mathbf{W}_g, \mathbf{c}_j, \sigma_j$ 四类参数；其中中心与宽度仅在对应该神经元被显著激活（ ϕ_j 大）时发生明显调整，体现 RBF 的局部学习特性，并使网络结构（基函数分布）随在线数据动态重构，从而提升逼近能力与控制鲁棒性。

4.2.6 控制器设计

使用 S-Function 模块实现 RBF 神经网络自适应控制器，设置输入端口为参考航向与实际航向，输出端口为控制输入，差分计算在 S-Function 内完成。设置权重学习率为 $\gamma_{wf} = 0.1$, $\gamma_{wg} = 0.1$ ，中心学习率为 $\gamma_c = 0.01$ ，宽度学习率为 $\gamma_\sigma = 0.01$ 。隐层神经元数设为 10，初始中心随机分布在 $[-1, 1]$ 内，初始宽度设置使用基于中心间距离的自适应启发式方法（k-NN 平均距离法），确保合理覆盖输入空间。

4.3 测试结果与分析

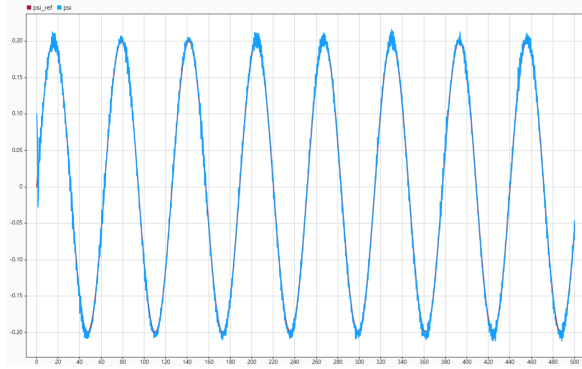


图 20: RBF 神经网络自适应控制结果

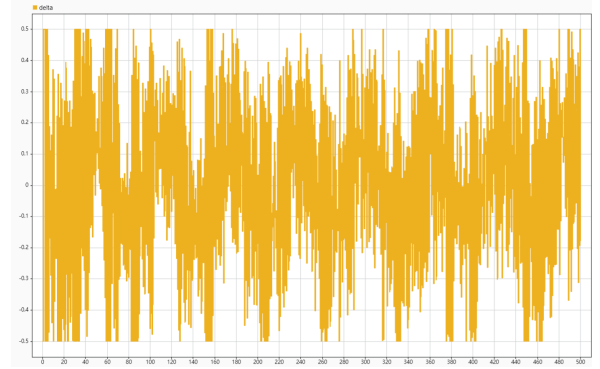


图 21: RBF 神经网络自适应控制输入

表 4: RBF 神经网络自适应控制性能指标汇总

指标	数值
最大跟踪误差 $e_{\max}$ (rad)	0.1000
均方根误差 e_{RMS} (rad)	0.0071
平均绝对误差 e_{MAE} (rad)	0.0052
归一化均方根误差 e_{nRMS}	0.0499
带内时间比例 R_{band}	99.85%
控制平滑性 S (rad/sample)	0.0097
控制 L_1 能量 E_1	98.9682
控制 L_2 能量 E_2	5.3948
控制变化率能量 V	773.0652

4.3.1 分析

从实验结果可以看出，基于 RBF 神经网络的自适应控制器在各项跟踪性能指标上均表现优异，尤其是在均方根误差和平均绝对误差方面，均显著低于前述直接神经网络控制和神经网络 PID 控制方法。此外，带内时间比例接近 100%，表明系统能够长时间保持在允许误差范围内。控制输入平滑性较好，变化率能量适中，显示出该控制器在保证高精度跟踪的同时，也具备良好的控制输入特性。

5 总结

本次实验设计并实现了三种基于神经网络的自适应控制方法，分别为直接神经网络控制、神经网络 PID 控制和 RBF 神经网络自适应控制。通过对比不同方法在动态参考信号跟踪任务中的性能指标，验证了各自的优缺点。直接神经网络控制方法简单易行，但在某些指标上表现稍逊；神经网络 PID 控制结合了传统 PID 的优势，适应性更强；而 RBF 神经网络自适应控制则在各项指标上均表现出色，显示出其在处理复杂非线性系统中的潜力。未来工作可进一步探索更复杂的神经网络结构和更高效的在线学习算法，以提升控制性能和系统鲁棒性。

参考文献

- [1] 课程讲义：专家控制（刘山浙江大学控制科学与工程学院）。
- [2] MATLAB Simulink 帮助文档。
- [3] CSDN 博客：径向基（Radial Basis Function:RBF）神经网络学习笔记 (https://blog.csdn.net/weixin_42555080/article/details/94661762)。

6 附录

6.1 文件清单

- system_identification.slx: USV 系统辨识 Simulink 模型。
- usv_direct_nn.slx: 直接神经网络控制 Simulink 模型。
- usv_nn_pid.slx: 神经网络 PID 控制 Simulink 模型。
- usv_rbf_adaptive.slx: RBF 神经网络自适应控制 Simulink 模型。
- test_model.slx: 系统辨识测试模型。
- identify.m: 系统辨识主程序。
- Direct_NN_Controller.m: 直接神经网络控制器 Level-2 S-Function 代码。
- NN_PID_Tuner.m: 神经网络 PID 控制器 Level-2 S-Function 代码。
- RBF_Adaptive_Controller.m: RBF 神经网络自适应控制器 Level-2 S-Function 代码。
- test_usv_net_function.m: USV 系统辨识模型仿真测试脚本。
- analyze_global.m: 全局性能指标计算脚本。
- generateUSVNetFunction.m: 生成 USV 系统辨识模型函数。
- *.png: 各类仿真结果图像文件。
- report.pdf: 实验报告 PDF 文件。